



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO



Ingegneria del Software

UML: Componenti **ITPS A.A. 2025/2026**

Vita Santa Barletta
Danilo Caivano
Antonio Piccinno

Dipartimento di Informatica - Università degli Studi di Bari
Via Orabona, 4 - 70125 - Bari
Tel: +39.080.5443270 | Fax: +39.080.5442536
serlab.di.uniba.it



Componenti

- **Definizione**

- ☐ Una parte del sistema che è **sostituibile**, **modulare**, che **incapsula** l'implementazione ed espone servizi

- **Esempi di componenti**

- ☐ File di tipo .exe, .bat, ...
- ☐ ActiveX Objects (OCX, ...)
- ☐ DLL (dynamic-link library)
- ☐ EJB (Enterprise JavaBeans)

- **Osservazione**

- ☐ Anche un **sottoinsieme di classi del sistema costituiscono una componente**, esponendo solo alcuni servizi
- ☐ Una componente può contenere altre componenti

Componenti

- Ogni componente è **indipendente**, quindi non interferisce con le operazioni di un altro componente
 - ❑ I dettagli dell'implementazione sono nascosti
 - ❑ L'implementazione di un componente può essere modificata senza influire sulle parti restanti del sistema
- I componenti comunicano attraverso **interfacce** ben definite
 - ❑ L'interfaccia del componente è espressa in termini di operazioni parametrizzate e il suo stato interno non viene mai mostrato
 - ❑ L'interfaccia dei **"servizi forniti"** definisce i servizi forniti dal componente
 - ❑ L'interfaccia dei **"servizi richiesti"** specifica i servizi che altri componenti del sistema devono fornire per far sì che il componente operi correttamente
- Le infrastrutture dei componenti forniscono una vasta gamma di servizi standard che possono essere utilizzati nei sistemi applicativi
 - ❑ Questo riduce la quantità di nuovo codice da sviluppare



Tipi di Componente

- **White-box**

- ☐ Si **conosce** sia il **funzionamento** della componente che la sua **implementazione**
- ☐ L'implementazione della componente **è aperta a modifiche**
- ☐ Esempio: un insieme delle classi del sistema

- **Black-box**

- ☐ Si **conosce** il **funzionamento** della componente ma **non** la sua **implementazione**
- ☐ L'implementazione della componente **non è aperta a modifiche**
- ☐ Esempi: librerie e framework proprietari, web service

- **Grey-box**

- ☐ Si **conosce** il **funzionamento** della componente e **non è necessario** (o **non si vuole**) conoscerne **l'implementazione**
- ☐ L'implementazione della componente **è aperta a modifiche**
- ☐ Esempio: librerie e framework open-source

Diagramma dei componenti

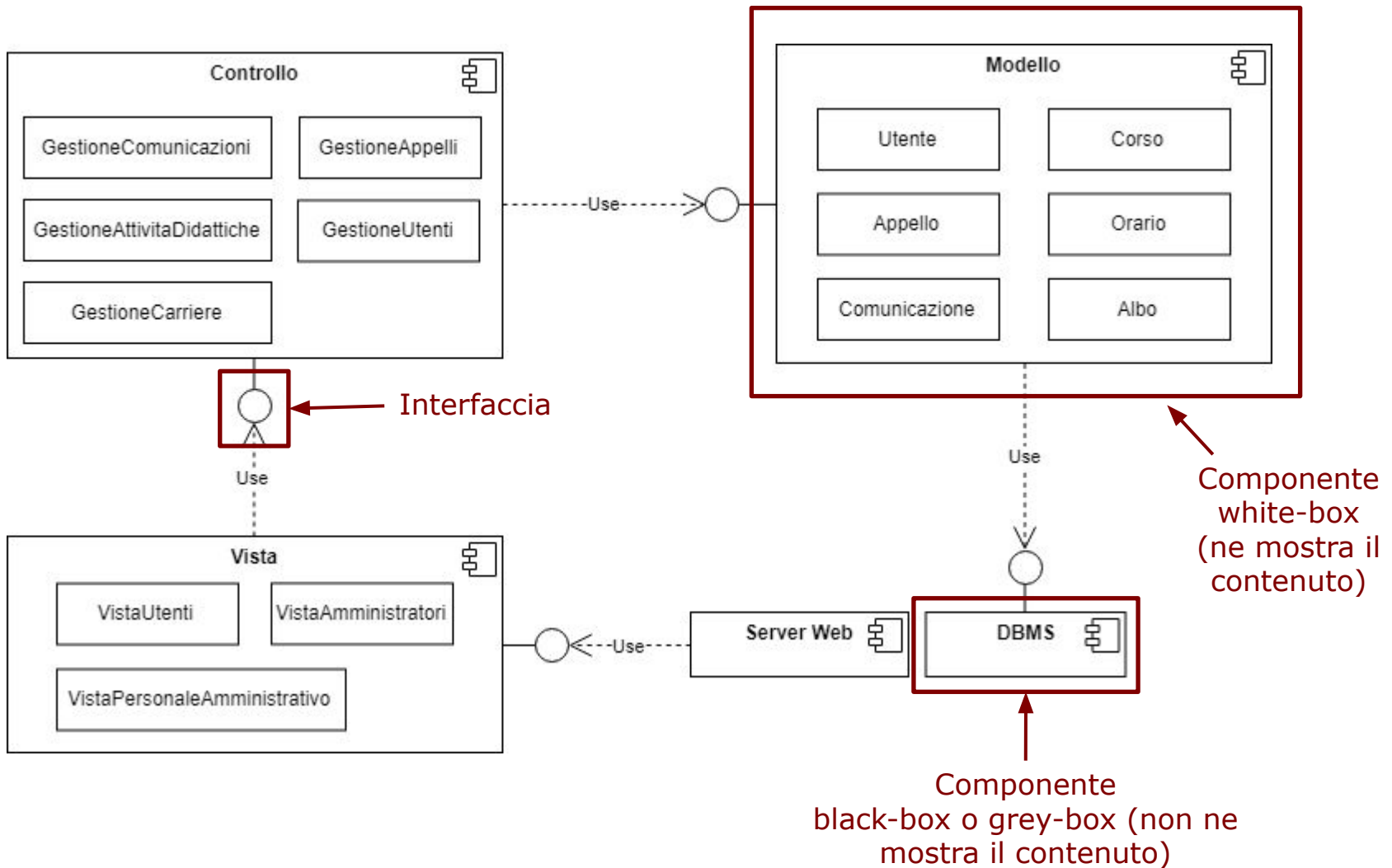
- **Definizione**

- ☐ Grafico che mostra le **componenti** e le **relazioni** di dipendenza tra queste

- **Osservazione**

- ☐ **Non mostra** le istanze delle componenti
- ☐ È possibile mostrare il **contenuto** di una componente (ad esempio le classi o le componenti che costituiscono quella componente)

Esempio: Diagramma dei componenti



Interfacce di un componente

Interfaccia dei servizi richiesti

Definisce i servizi che sono richiesti e che devono essere forniti da altri componenti



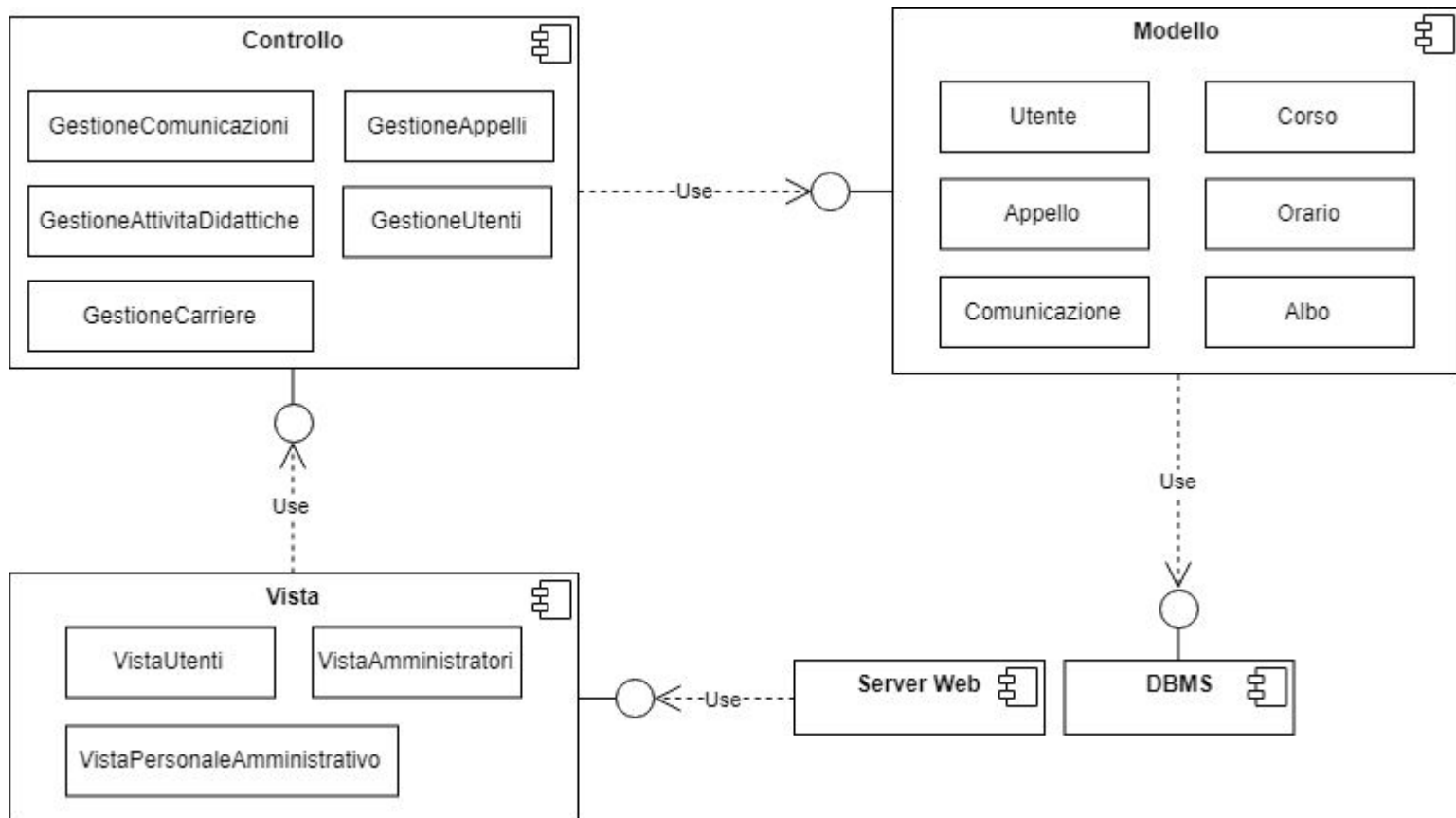
Interfaccia dei servizi forniti

Definisce i servizi che sono forniti dal componente ad altri componenti

- L'interfaccia dei **"servizi forniti"** è l'API del componente; definisce i metodi che possono essere chiamati da un utente del componente
- L'interfaccia dei **"servizi richiesti"** che sono richiesti
 - ❑ Se questi servizi non sono disponibili, il componente non potrà funzionare
 - ❑ Questo non compromette l'indipendenza o la consegna del componente, perché l'interfaccia dei servizi richiesti non definisce come questi servizi devono essere forniti

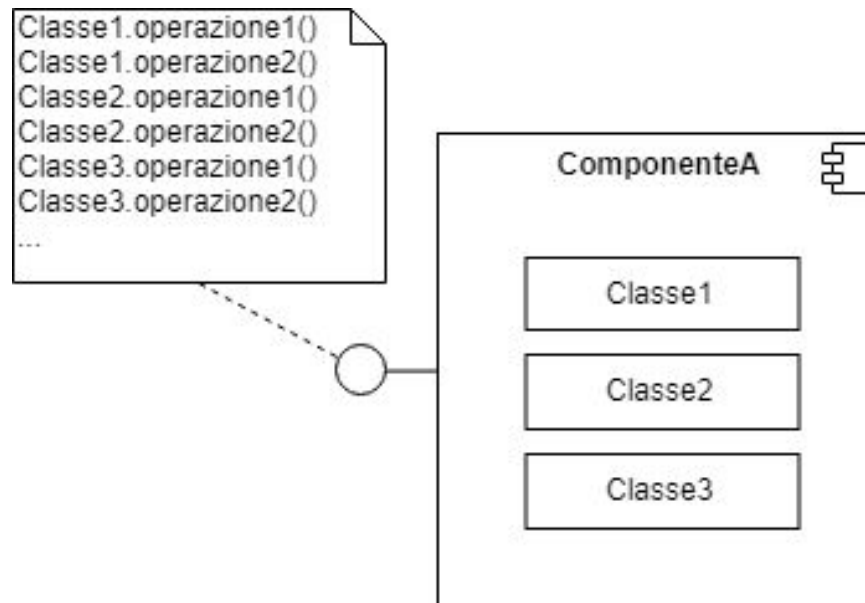


PROBLEMA: Quali operazioni espone ciascuna componente tramite la sua interfaccia?



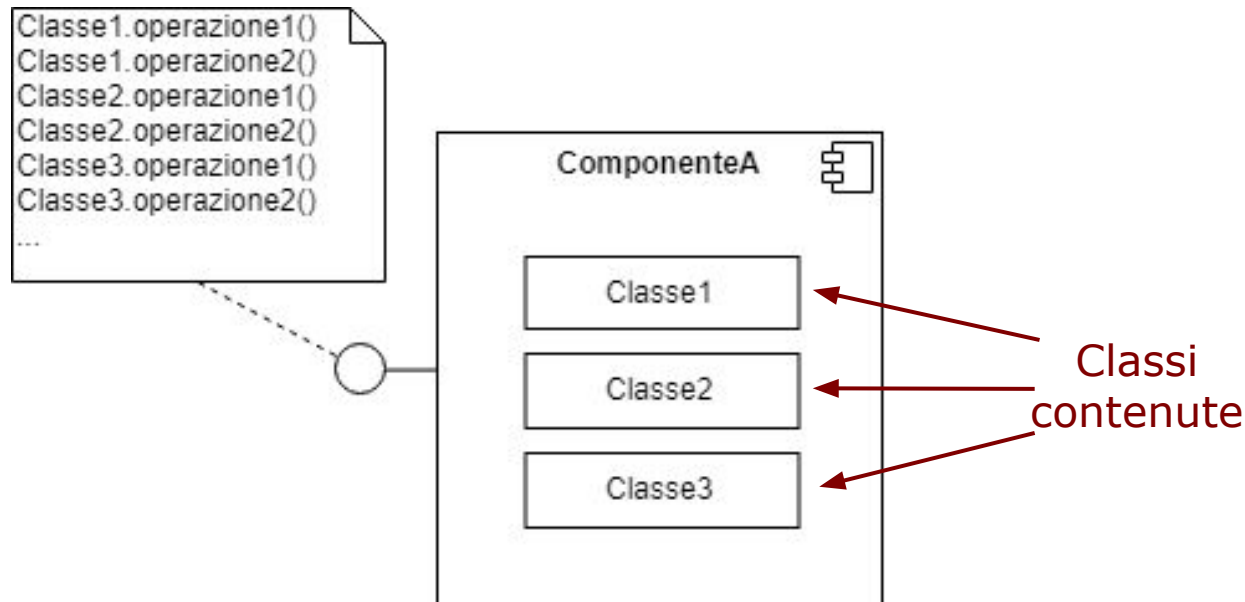
Soluzione

- **Usare le annotazioni** per specificare quali operazioni vengono esposte dall'interfaccia
 - ❑ Per le componenti white-box indicare anche le classi che forniscono queste operazioni



Mapping tra classi e componenti

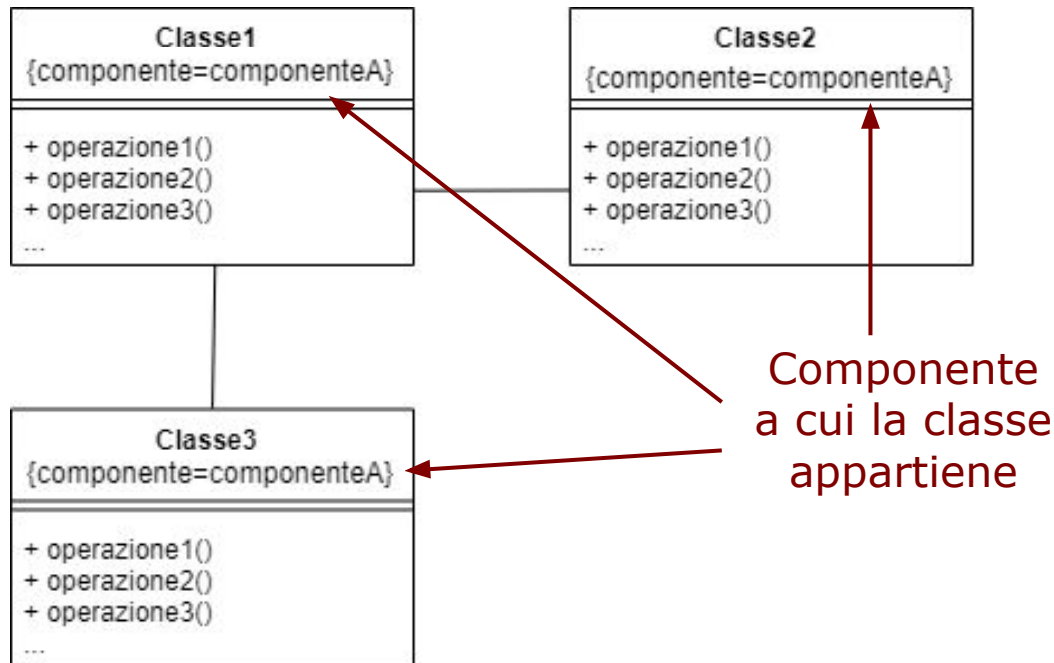
- **Nel diagramma delle componenti**
 - Mostrare le classi contenute in ciascuna componente (white-box)



Mapping tra classi e componenti

- **Nel diagramma delle classi**

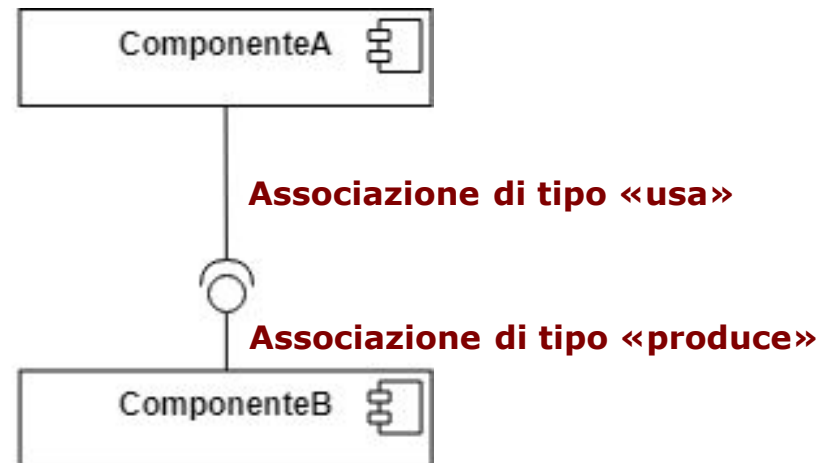
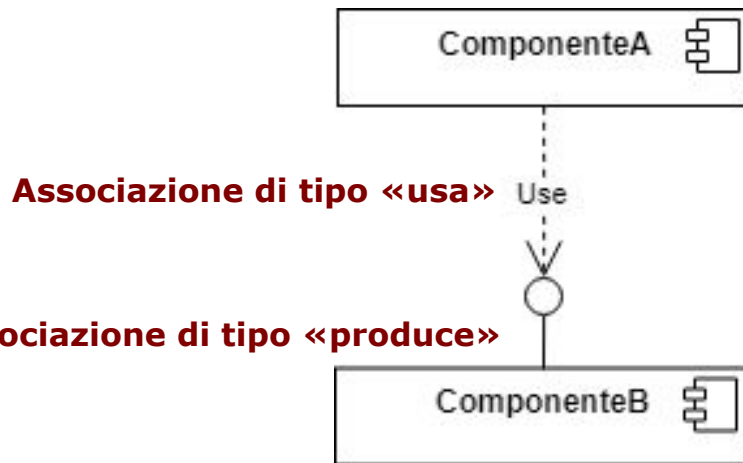
- ❑ Mostrare la componente a cui ciascuna classe appartiene



Notazioni equivalenti

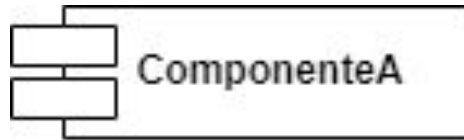
- **Dipendenza d'uso**

- ❑ La notazione di destra equivale a quella di sinistra



Notazioni equivalenti

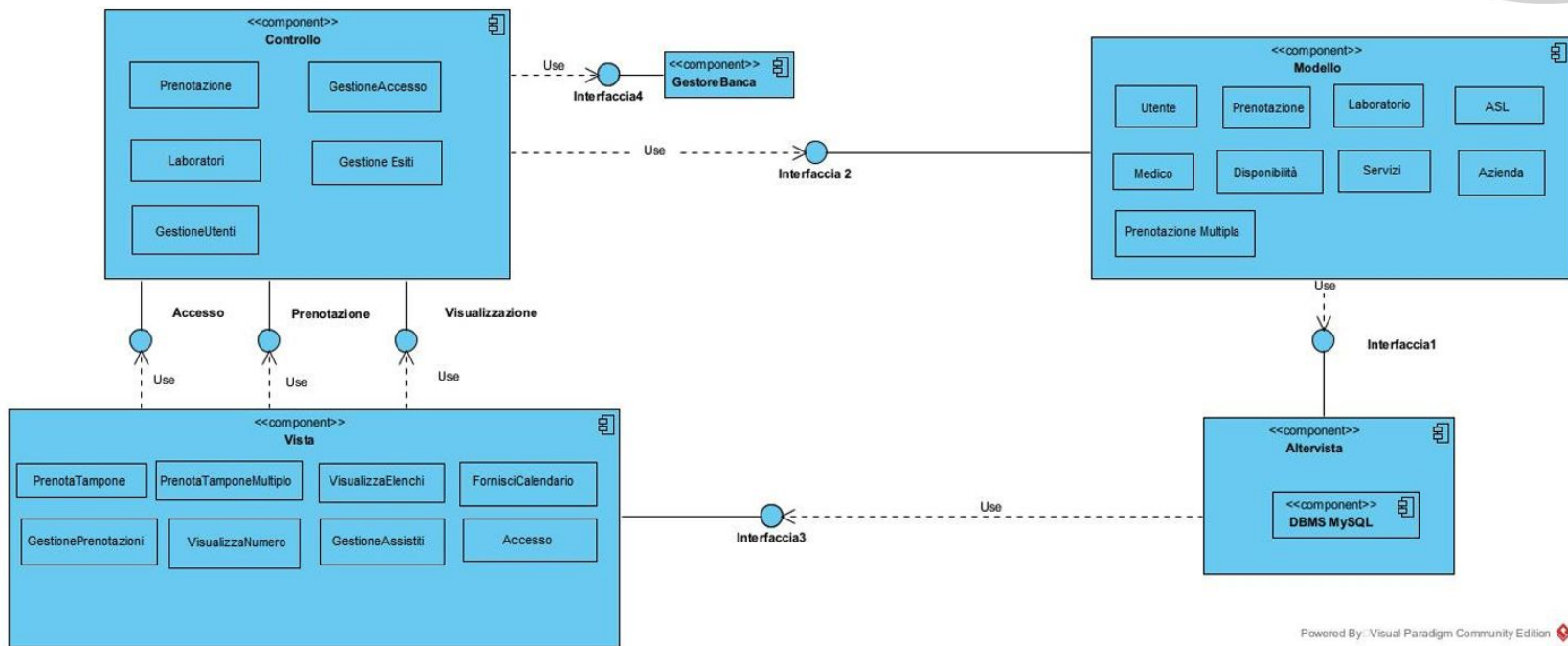
- **Componente**
 - La notazione di destra equivale a quella di sinistra



Esempio diagramma delle componenti



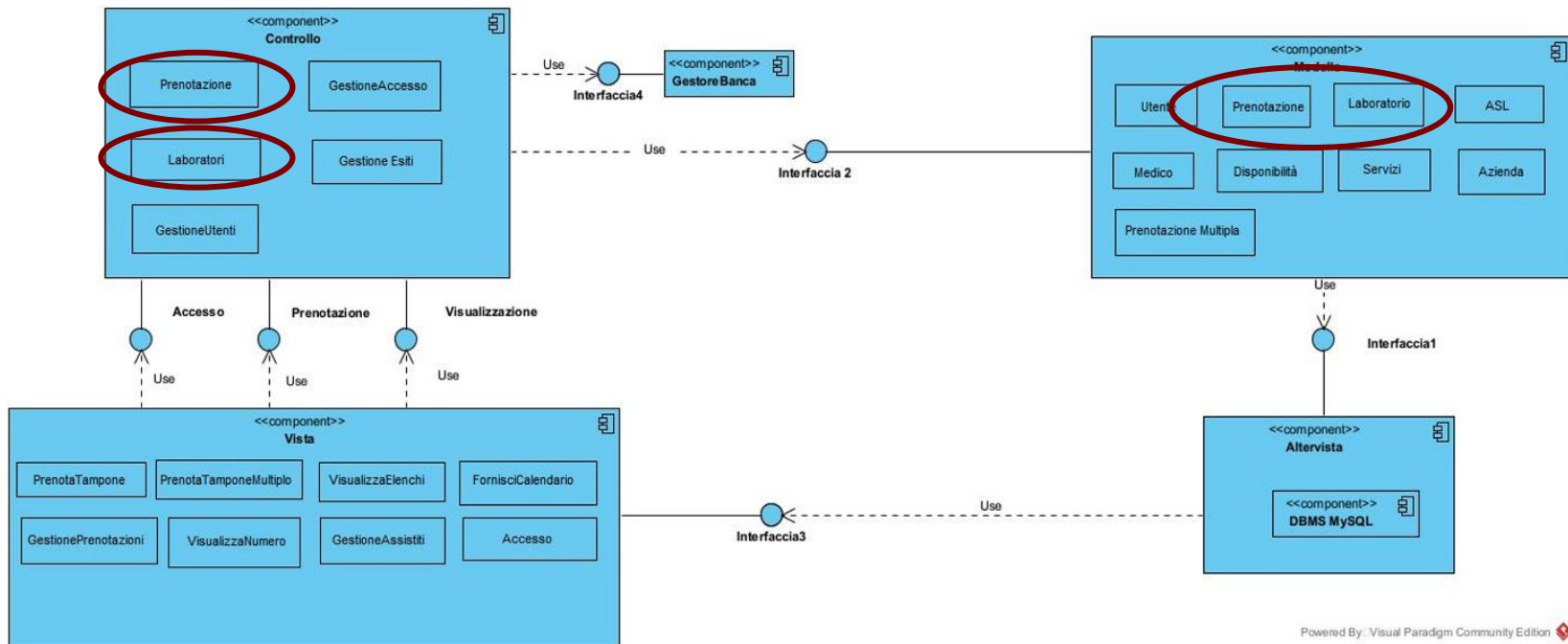
- Caso di studio a.a.2020/2021



Esempio diagramma delle componenti



- Caso di studio a.a.2020/2021



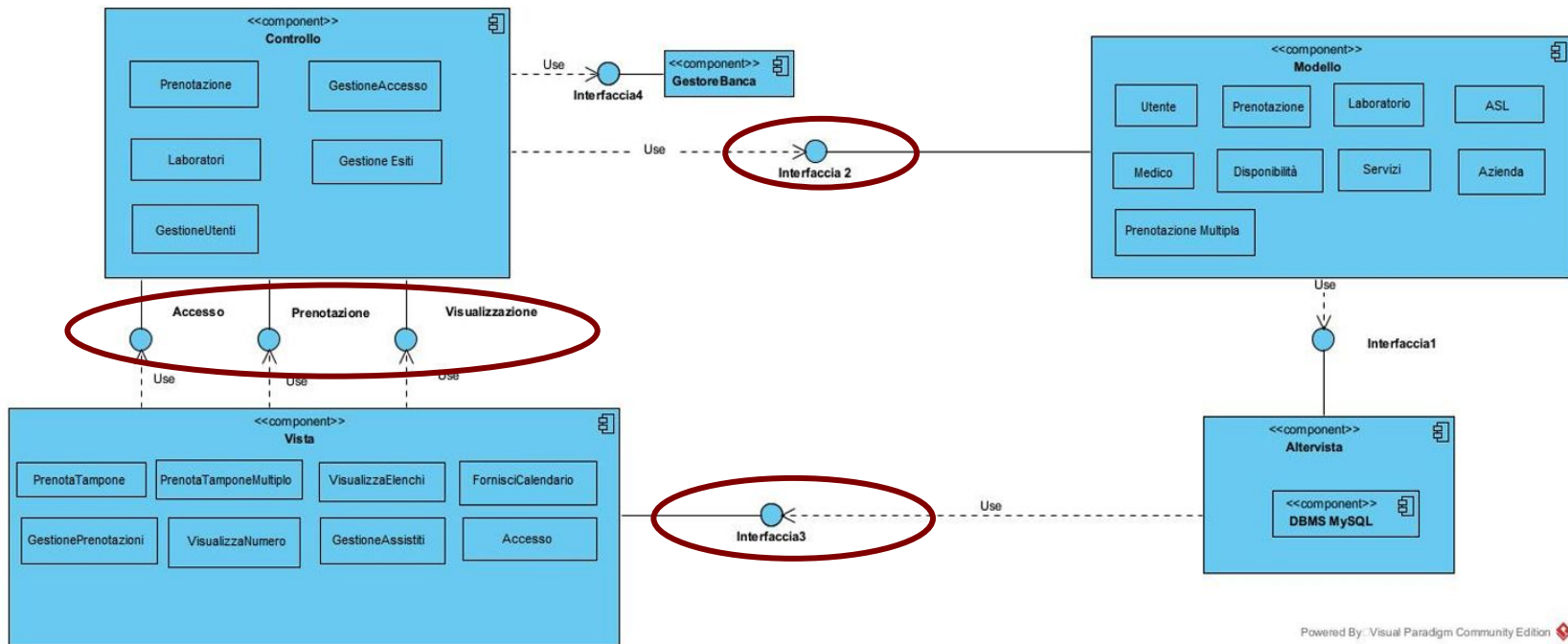
Powered By: Visual Paradigm Community Edition



Esempio diagramma delle componenti



- Caso di studio a.a.2020/2021



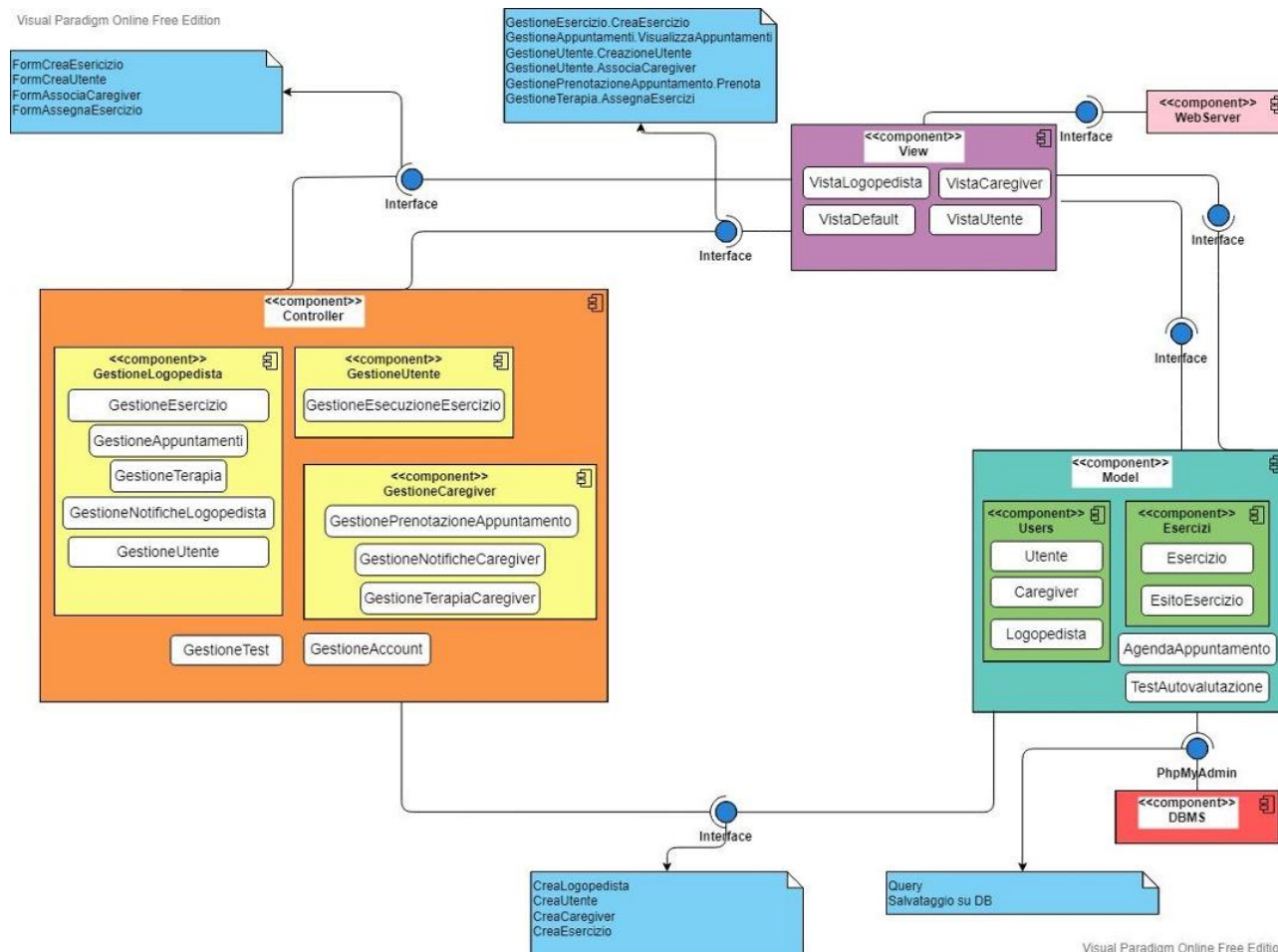
Powered By: Visual Paradigm Community Edition



Esempio diagramma delle componenti



- Caso di studio a.a.2021/2022



Riferimenti

- Ian Sommerville “Ingegneria del Software”, 10a ed. Pearson, 2017
 - Capitolo 16: Ingegneria del software basato sui componenti
 - 16.1 Componenti e modelli di componenti

Ulteriori testi consigliati

- Martin Fowler, “UML DISTILLED, 4a edizione” Pearson addison Wesley, 2010
 - Capitolo 14: Diagramma delle componenti

